SimQC – Weekly Progress Report

Aoutul Nabi Purna*, Shishir Dhar†, Shihab Sharar‡, and Md. Iftikhar Alam Hridik§
Department of Electrical and Computer Engineering, North South University, Dhaka, Bangladesh
Emails: {*aoutul.purna.241, †shishir.dhar.241, ‡shihab.sharar.241, §iftikhar.hridik.241}@northsouth.edu

***Abstract*—This report outlines the foundational development of the SimQC simulation engine, focusing on the mathematical abstraction layer, memory layout decisions, and baseline gate-application benchmarks.**

I. Progress Summary

During this period, we completed the mathematical foundation layer of the simulation core and prepared the architecture for the state-vector engine[cite: 1]. Completed deliverables include:

1) A `Complex` class implementing addition, multiplication, conjugation, modulus, and scalar helpers, with unit tests validated against exact analytical values (proposal validation Layer 1)[cite: 1];
2) A `Matrix` class supporting complex matrix-vector and matrix-matrix products, which will double as the naive $\mathcal{O}(4^n)$ reference implementation for differential testing (Layer 6)[cite: 1];
3) A written conventions specification (`CONVENTIONS.md`) fixing bit ordering, memory layout, and the gate-application contract before any state-vector code is written[cite: 1].

We deliberately deferred implementation of the `StateVector` class until conventions were frozen, since endianness and layout decisions propagate into every subsequent kernel[cite: 1].

II. Learning: What to Use, What Not to Use, and Why

The textbook Kronecker product approach is unusable at scale[cite: 1]. Lifting a gate G on qubit t to the full $2^n \times 2^n$ operator:

$$U = I \otimes \cdots \otimes G \otimes \cdots \otimes I \tag{1}$$

and multiplying it against the state vector costs $\mathcal{O}(4^n)$[cite: 1]. At 16 qubits, a single gate matrix requires $\sim$34 GB of RAM[cite: 1]. Furthermore, the matrix is overwhelmingly sparse; for $(I \otimes I \otimes H)$ on 3 qubits, 48 of the 64 entries are zero, meaning 75% of the arithmetic involves zero amplitudes[cite: 1].

The matrix-free pairwise method (Eq. 6 of our proposal) instead updates only the 2^{n-1} amplitude pairs whose indices differ in bit t ($j = i \oplus 2^t$), each with the same 2×2 matrix G, yielding $\mathcal{O}(2^n)$ runtime per gate[cite: 1]. Our worked 3-qubit example confirmed that the two methods are numerically identical and that a CNOT gate reduces further to pure in-place amplitude swapping (a permutation), requiring no floating-point multiplication[cite: 1].

Entanglement forbids lazy Kronecker products. We initially considered keeping each qubit as an independent 2×1 vector and deferring the tensor product to the end[cite: 1]. This holds only while the state is separable: the separability condition for a 2-qubit state,

$$c_{00}c_{11} - c_{01}c_{10} = 0, \tag{2}$$

holds after single-qubit gates but is violated the moment an entangling gate (e.g., CNOT, CZ) acts[cite: 1]. CNOT itself does not factorize as $A \otimes B$[cite: 1]. Simulators must therefore maintain the full 2^n-dimensional vector from the start, matching the memory constraint (warning threshold at 20 qubits, hard cap at 24 qubits) specified in our project objectives[cite: 1].

Object allocation does not belong in the hot path. Although our `Complex` class is complete and correct, representing the state vector as an array of `Complex` objects was rejected[cite: 1]. In V8, each object overhead costs roughly 40–56 bytes, compared to 16 bytes for two 64-bit floats in a `Float64Array` ($\approx 3\times$ memory reduction)[cite: 1]. Additionally, complex multiply-add operations with objects allocate temporary heap instances; at 2^{20} amplitudes per gate, this triggers garbage-collector pauses that freeze the editor UI during drag-and-drop interactions[cite: 1].

Decision: The `StateVector` class exposes a clean facade (`get`/`set` by index) while storing amplitudes in two flat `Float64Array` buffers (`re`, `im`)[cite: 1]. The `Complex` and `Matrix` classes remain restricted to gate construction (flattened to raw number tuples once at definition time), test fixtures, and UI calculations[cite: 1]. The pairwise execution kernel operates exclusively on primitive doubles and performs zero heap allocation[cite: 1].

III. Conventions Adopted

Table I records the design decisions frozen in `CONVENTIONS.md` before implementation of the state-vector code[cite: 1].

IV. Literature Review: Conventions in Existing Systems

Conventions are neither standardized nor incidental; divergence between implementations is a documented source of cross-library bugs[cite: 1]. Qiskit [2] uses little-endian qubit ordering internally (qubit 0 = least significant bit), matching our indexing scheme[cite: 1]. However, Qiskit circuit diagrams display qubit 0 at the top, creating a divergence between display and internal index representations[cite: 1]. We adopt

TABLE I
CONVENTIONS ADOPTED FOR THE SIMULATION CORE (FROZEN IN CONVENTIONS.MD)

Convention	Decision
Qubit ordering	Little-endian: qubit q = bit q of the amplitude index (partner index is $j = i \oplus (1 \ll q)$); UI reverses bits for display only.[cite: 2]
Memory layout	Two `Float64Array` instances, `re[i]` and `im[i]`, hidden behind the `StateVector` facade.[cite: 2]
Kernel arithmetic	Raw scalar doubles only; no `Complex` instances inside gate-application loops.[cite: 2]
Controlled gates	Pair update applied only when `(i & controlMask) === controlMask`; non-matching amplitudes left untouched bit-for-bit.[cite: 2]
Normalization	Norm verified after every gate; renormalize only if numerical drift $> 10^{-9}$, logging a warning (silent corrections mask arithmetic bugs).[cite: 2]
Column semantics	Gates within a single circuit column must act on disjoint qubits; overlap is validated and prevented in the UI.[cite: 2]
Serialization	JSON schema stores explicit qubit indices; endianness is explicitly declared in the schema metadata.[cite: 2]

this separation explicitly (the core computes via indices, while the UI formats via labels)[cite: 1].

OpenQASM [5] orders qubits as declared in `qreg` statements and remains agnostic regarding simulator internal layouts, prompting our JSON schema to declare endianness explicitly[cite: 1]. Quirk [1] avoids exposing indexing conventions to users by presenting per-column probability and amplitude displays instead of raw basis labels, a choice suited for real-time reactive inspection[cite: 1, 2].

High-performance simulation engines such as qHiPSTER [4] and the parallel framework of De Raedt et al. [3] employ the same pairwise amplitude update model used here, establishing that $\mathcal{O}(2^n)$ in-place updates over an explicit state vector represent standard practice[cite: 2]. Our differential test harness (Layer 6) is designed to mitigate cross-framework errors caused by index and convention mismatches[cite: 2].

V. PROPOSED SOLUTION AND NEXT STEPS

During the next sprint, we will execute the following tasks:

1) Implement the `StateVector` class (split-array layout, facade API, zero-state initialization $\alpha_0 = 1$)[cite: 2];
2) Implement the pairwise update kernel with block iteration (stride 2^t, branch-free inner loop, unrolled 2×2 complex multiply-add, and bitwise control masks)[cite: 2];
3) Define single-qubit gates $(X, Y, Z, H, R_x, R_y, R_z)$ via `Matrix` representations flattened into raw numeric tuples[cite: 2];
4) Construct the differential testing harness comparing the pairwise kernel against the naive matrix product path for $n \leq 6$ over pseudo-random state distributions[cite: 2].

Exit criterion: Producing a Bell state via H followed by CNOT must yield normalized amplitudes $(1/\sqrt{2}, 0, 0, 1/\sqrt{2}) \approx (0.7071, 0, 0, 0.7071)$ up to endianness conventions, with naive-versus-kernel agreement maintained to a tolerance of 10^{-12}[cite: 2].

VI. AI MODULE (EXPLAINER & BUILDER)—PHASED IMPLEMENTATION

A. Rationale: Pretrained Open-Weights Models, Not From-Scratch Training

Training a language model from scratch is infeasible at course scale, requiring compute and data budgets orders of magnitude beyond our scope[cite: 3]. We instead adopt the established practice of deploying a *pretrained, openly licensed instruct-tuned model* and adapting it to our domain[cite: 3]. Adaptation proceeds along a ladder of increasingly involved techniques—prompting, grounding, few-shot exemplars, evaluation, and finally parameter-efficient fine-tuning (LoRA [9], QLoRA [10])—with each rung independently useful if the project stops there[cite: 3]. A key architectural insight motivates this design: our simulation core already computes the exact intermediate state after every circuit column[cite: 3]. The core therefore serves as a *ground-truth labeling oracle*, both for grounding explainer prompts in real computed states and for synthetically generating the fine-tuning dataset (Phase 5 of our plan), eliminating manual annotation[cite: 3].

B. Phase 0—Conceptual Foundation

Before any integration code, the team is completing a structured study of: (i) tokenization and autoregressive inference [8]; (ii) base versus instruct-tuned models; (iii) post-training quantization (e.g., the GGUF `Q4_K_M` format, ≈4.5 bits/weight) and its memory–quality trade-off; and (iv) the three-layer local stack: the `llama.cpp` inference engine [11], the GGUF weight format, and Ollama [6] as the serving layer exposing a REST API on `localhost:11434`[cite: 3]. Deliverables for this phase are an internal comprehension checklist and a frozen interface contract (`AI_INTERFACE.md`), mirroring the conventions-first discipline applied to the simulation core in Section III[cite: 3].

C. Phase 1—Local Inference Infrastructure and Hardware Budget

Ollama has been installed and candidate models evaluated against the team's deployment hardware (16 GB sys-

tem RAM, RTX 3050 Laptop GPU with 4 GB VRAM, Intel Core i5 12th-gen)[cite: 3]. The constraint analysis is summarized in Table II[cite: 3]. *Decision:* `qwen2.5:3b` serves as the default development model (loads entirely into VRAM, 100% GPU execution, ≈40 tokens/s measured), while `qwen2.5-coder:7b` is reserved for quality-reference runs via hybrid CPU/GPU offload[cite: 3]. The 14B tier exceeds the local budget and is deferred to a cloud notebook (Google Colab) as the teacher model for Phase 5 dataset generation[cite: 3]. Two integration constraints were identified: Ollama's default 2048-token context window must be raised for our circuit-JSON prompts, and browser cross-origin requests require `OLLAMA_ORIGINS` to be set during development[cite: 3]. A measured throughput baseline for each candidate is recorded in the project log and will serve as the comparison point for the fine-tuned model in Phase 7[cite: 3].

TABLE II
CANDIDATE MODEL EVALUATION UNDER HARDWARE BUDGET (16 GB RAM, RTX 3050 4 GB VRAM, GGUF Q4_K_M QUANTIZATIONS)

Model (Ollama tag)	Disk	Runtime Mem.	Hardware Fit	Role
`qwen2.5:3b`	1.9 GB	≈3 GB	100% GPU	Default dev explainer[cite: 3]
`qwen2.5-coder:7b`	4.7 GB	≈6 GB	≈46/54 CPU/GPU	Quality reference[cite: 3]
`qwen2.5:14b`	9.0 GB	≈11 GB	Exceeds budget	Phase-5 teacher (Colab)[cite: 3]

VII. RISKS AND MITIGATIONS

1) **Memory Ceiling:** At 24 qubits, amplitudes require ≈256 MB of RAM[cite: 2]. Mitigated by enforcing a hard simulation limit at 24 qubits with UI threshold warnings at 20 qubits[cite: 2].
2) **GC Pressure:** Mitigated by the zero-allocation kernel design detailed in Section II[cite: 2].
3) **Bit-Ordering Inconsistencies:** Mitigated by the frozen conventions specification (Section III) and automated differential tests against the reference implementation[cite: 2]. For analytical iteration structures, refer to Press et al. [7][cite: 2].

REFERENCES

[1] C. Gidney, "Quirk: A drag-and-drop quantum circuit simulator that runs in your browser," GitHub repository, Apache-2.0 license. [Online]. Available: https://github.com/strilanc/quirk

[2] A. Javadi-Abhari *et al.*, "Qiskit: An open-source framework for quantum computing," IBM, 2017. [Online]. Available: https://qiskit.org

[3] H. De Raedt and K. Michielsen, "Computational methods for simulating quantum computers," in *Quantum Computing and Quantum Bits in Mesoscopic Systems*. Hoboken, NJ, USA: Wiley, 2004.

[4] M. Smelyanskiy, N. P. D. Sawaya, and A. Aspuru-Guzik, "qHiPSTER: The Quantum High Performance Software Toolkit," 2016, *arXiv:1601.07195*.

[5] A. W. Cross *et al.*, "OpenQASM 3 specification," 2021. [Online]. Available: https://openqasm.com

[6] Ollama, "Run large language models locally," 2023. [Online]. Available: https://ollama.com

[7] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed. Cambridge, UK: Cambridge Univ. Press, 2007.

[8] A. Karpathy, "Intro to large language models," YouTube lecture, 2023. [Online]. Available: https://youtu.be/zjkBMFhNj_g

[9] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," 2021, *arXiv:2106.09685*.

[10] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," 2023, *arXiv:2305.14314*.

[11] G. Gerganov, "llama.cpp: Port of Facebook's LLaMA model in C/C++," GitHub repository, 2023. [Online]. Available: https://github.com/ggerganov/llama.cpp